Hyperparameter Tuning & Cross-Validation

The Complete Guide — From Default Models to Optimised, Honest Performance

> *"A model with the wrong hyperparameters is like a student who studied the wrong chapters. Cross-validation is the mock exam that tells you — honestly — whether you are ready."*

How to Use This Guide

This guide uses **one real-world scenario throughout every section** so every concept is grounded in something tangible.

> 🏥 **Running Example — Hospital Diabetes Prediction** You have data on **800 training patients** with features: `Age` | `BMI` | `Blood Pressure` | `Glucose` | `Insulin` Target: **Diabetic (1) or Not Diabetic (0)**
>
> You are trying three models: Logistic Regression, KNN, and SVM. Each has hyperparameters. Each can underfit or overfit. Cross-validation will find the honest best settings. The test set (200 patients) stays **locked** until the very end.

PART 1 — What Are Hyperparameters?

1.1 Parameters vs. Hyperparameters — The Fundamental Distinction

Before understanding tuning, you must understand what you are and are not tuning.

Parameters — Learned From Data

Parameters are the **internal values** that a model learns during training by minimising a loss function. The optimiser (Gradient Descent) adjusts them automatically. You never set them manually.

```
Linear / Logistic Regression:
    Parameters = weights w1, w2, w3... and bias b
    These are updated by gradient descent every iteration.

SVM:
    Parameters = the weight vector w and bias b defining the hyperplane
    These are determined by the quadratic programming solver.

KNN:
    Parameters = NONE
    KNN is a lazy learner — it has no parameters to learn at all.
```

Hyperparameters — Set Before Training

Hyperparameters are the **settings that control the learning process itself**. No gradient, no solver, no training loop can find them — they sit *outside* the training loop. You must choose them before training begins.

```
Logistic Regression:
    Hyperparameters = C (regularisation strength), penalty type (L1 or L2)

KNN:
    Hyperparameters = K (number of neighbors), distance metric

SVM:
    Hyperparameters = C (margin hardness), gamma (kernel width), kernel type

Decision Tree:
    Hyperparameters = max_depth, min_samples_split, min_samples_leaf

Neural Network:
    Hyperparameters = learning rate, number of layers, neurons per layer,
                      dropout rate, batch size, number of epochs
```

The key difference in one sentence:

> Parameters are *discovered* by the algorithm from data. Hyperparameters are *chosen* by you, before the algorithm even starts.

1.2 Why Hyperparameters Cannot Be Learned From Data

A natural question: "Why can't we just add hyperparameters to the gradient descent loop and optimise them too?"

The answer is that hyperparameters operate at a **different level** of the learning process:

- `C` in Logistic Regression determines *how strongly the loss function penalises large weights* — it shapes the loss landscape that gradient descent navigates. You cannot put C inside the gradient descent loop because C defines the rules of that loop.
- `K` in KNN determines *how many neighbors participate in a vote* — there is no loss function and no gradient here at all. K determines the entire decision mechanism.
- `max_depth` in a Decision Tree determines *when the tree stops splitting* — it is a stopping criterion, not a parameter being optimised.

Think of it this way:

```
HYPERPARAMETERS = The rules of the game
PARAMETERS      = How well you play within those rules

You cannot learn the rules of chess by playing chess.
```

```
You have to be told the rules (hyperparameters) first.
Then you can get better at playing (parameters optimised during training).
```

1.3 The Model Complexity Dial

Every hyperparameter, regardless of the algorithm, ultimately controls one thing: **model complexity**.

```
                    MODEL COMPLEXITY SPECTRUM

← LOW COMPLEXITY                                        HIGH COMPLEXITY →
  (Simple model)                                        (Complex model)

  Logistic Reg      Logistic Reg      Logistic Reg
  C = 0.001         C = 1.0 ✅         C = 1000
  Underfits         Sweet spot        Overfits

  KNN               KNN               KNN
  K = 500           K = 11 ✅          K = 1
  Underfits         Sweet spot        Overfits

  SVM               SVM               SVM
  Low C, Low γ      C=1, γ=0.01 ✅     High C, High γ
  Underfits         Sweet spot        Overfits
```

Every tuning exercise is fundamentally asking: "Where on this complexity spectrum does my specific dataset live?"

The answer depends on:

- How much training data you have (more data → can support higher complexity)
- How noisy the data is (more noise → lower complexity is safer)
- How many features you have (more features → risk of overfitting increases)
- The true complexity of the underlying pattern (simple pattern → simple model)

PART 2 — The Root Problem: Why Default Models Fail

2.1 Underfitting — The Model Is Too Simple

When the model is too simple for the complexity of the data, it fails to capture the true pattern. It performs badly on **both training and test data**.

```
SCENARIO: Logistic Regression with C = 0.0001 (extreme regularisation)

Training Accuracy:  62%
Test Accuracy:      60%
```

```
Gap:  2%   ← small gap, but BOTH are low → Underfitting
```

What is happening internally: The regularisation term is so large that it forces nearly all weights toward zero. The model can barely distinguish diabetic from non-diabetic patients — it essentially predicts the majority class for everyone.

Visualising underfitting in KNN:

```
K = 500 (half the training set votes for every prediction):

Patient space:
  D D D D D D     N N N N N N
  D D D D D D     N N N N N N
  D D D D D D     N N N N N N

  Boundary: ──────────────────── (rough center)

The boundary is a crude straight line.
It ignores all the local cluster structure.
50% of D-region patients and 50% of N-region patients are in each territory.
```

Analogy: A student who studied only the chapter titles — they have a rough idea of what the exam is about but miss every specific question.

2.2 Overfitting — The Model Is Too Complex

When the model is too complex for the amount of data, it memorises the training data — including its noise and quirks — and fails to generalise to new data.

```
SCENARIO: KNN with K = 1

Training Accuracy:  99%    ← looks incredible!
Test Accuracy:      63%    ← falls apart on new patients

Gap:  36%  ← large gap → Overfitting
```

What is happening internally: With K=1, every training patient is classified by only its single nearest neighbor — which is itself. So every training point is always predicted correctly (100% training accuracy is possible). But any new patient is assigned to the class of whichever single training patient happens to be closest — highly sensitive to noise.

Visualising overfitting in KNN:

```
K = 1:

Patient space:
  D D N D D D     N D N N N N
  D D D D N D     N N N N N N
  D D D D D D     D N N N N N
        ↑               ↑
   Noisy N patient    Noisy D patient
   surrounded by D    surrounded by N

The boundary is jagged, wrapping around every individual point.
It has memorised the noise (the wrong-label patients)
rather than the underlying pattern.
```

Analogy: A student who memorised the exact questions from last year's practice paper. They ace the practice paper (training data) but fail when the real exam has different questions (test data).

2.3 The Sweet Spot — Low Bias, Low Variance

```
SCENARIO: SVM with C = 1.0, γ = 0.01 (after tuning)

Training Accuracy:  89%
Test Accuracy:      87%

Gap:  2%   ← small gap, BOTH are high → Sweet Spot ✅
```

The model has learned the **true underlying pattern** — the genuine relationship between BMI, Glucose, Age, and diabetes risk — without memorising the idiosyncrasies of the 800 specific training patients.

```
THE DIAGNOSTIC TABLE

Training Acc    Test Acc    Gap     Diagnosis
    Low            Low      Small   HIGH BIAS → Underfitting
    High           Low      Large   HIGH VARIANCE → Overfitting
    High           High     Small   SWEET SPOT ✅
    ~50%           ~50%     ~0%     Model is not learning at all
```

2.4 The Bias-Variance Tradeoff and Hyperparameters

This is the mathematical foundation of why hyperparameter tuning works.

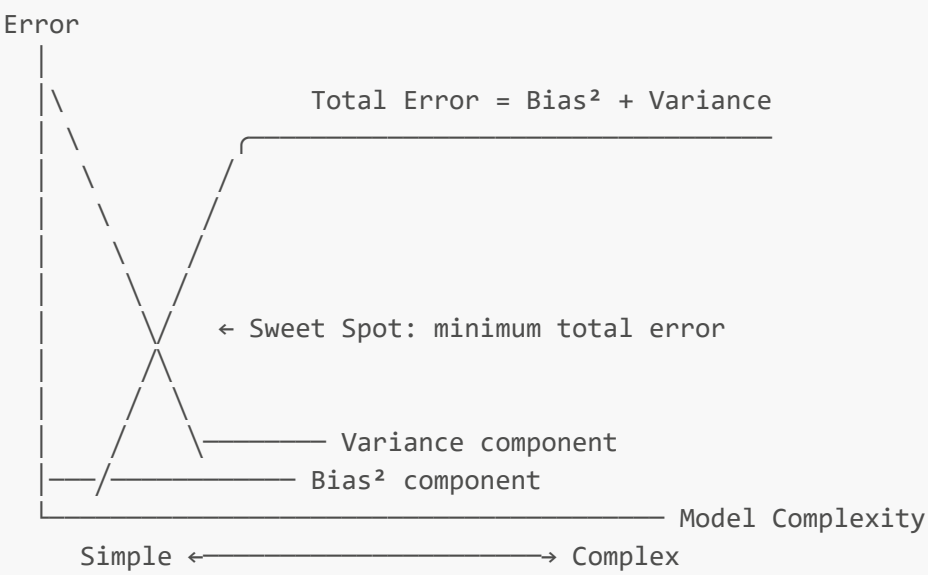
```
Total Error  =  Bias²  +  Variance  +  Irreducible Noise
```

Bias: Error from the model's assumptions being too restrictive. A model with high bias *cannot* capture the true pattern, no matter how much data you give it.

Variance: Error from the model being too sensitive to the specific training data. A model with high variance would give completely different predictions if trained on a slightly different set of 800 patients.

Irreducible Noise: Randomness in the real world no model can eliminate. Some patients with identical features will have different outcomes — this is irreducible.

The **tradeoff** is that every step you take to reduce Bias (make the model more complex) automatically increases Variance, and vice versa:



The goal of hyperparameter tuning is to find the bottom of this U-curve.

Cross-validation is how you measure your Y-axis position (your total error) for each X-axis position (each complexity/hyperparameter setting) — without being fooled by the test set.

PART 3 — Hyperparameters Deep Dive: Algorithm by Algorithm

3.1 Logistic Regression — C (Regularisation Strength)

What Regularisation Does

Without regularisation, Logistic Regression can assign enormous weights to any feature that helps separate training patients — even if that feature's relationship is just coincidental noise.

```
Unregularised model on 800 training patients:
w_Glucose  = +8.4
w_BMI      = +6.2
w_Age      = +0.3
```

```
w_feature7 = -12.3   ← this feature has a coincidental pattern in training data

When new patients arrive, this pattern doesn't hold → model fails badly.
```

Regularisation adds a **penalty term** to the loss function that discourages large weights:

```
Loss  =  Log_Loss  +  (1/C) * ||w||²

where  ||w||²  =  w₁² + w₂² + w₃² + ...   (sum of squared weights)
```

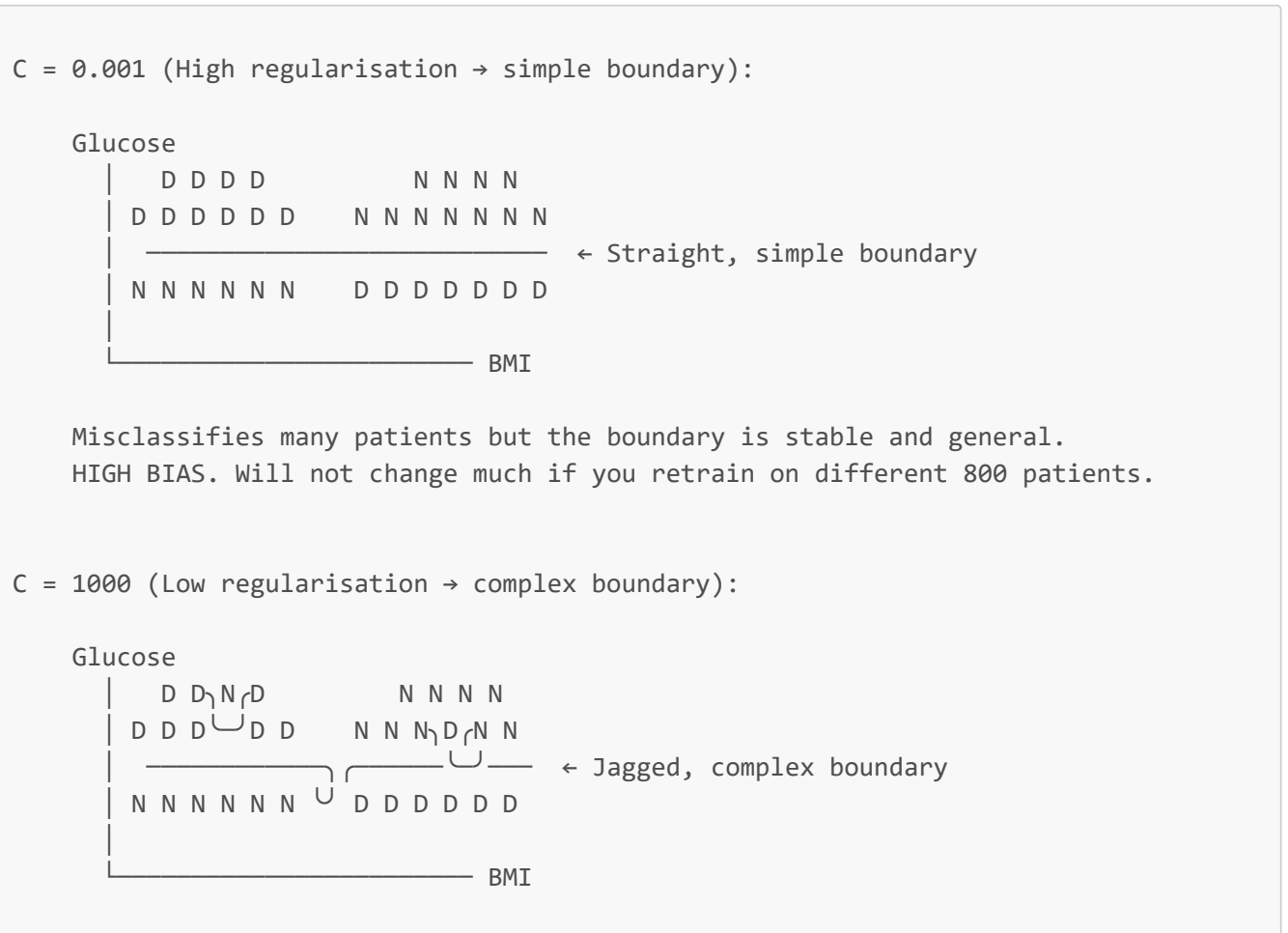
C is the inverse of regularisation strength:

```
Large C (e.g., C = 1000):
    (1/C) = 0.001  → penalty is tiny → weights can grow large → overfitting

Small C (e.g., C = 0.001):
    (1/C) = 1000   → penalty is huge → weights forced to ~0 → underfitting

C = 1.0 (default):
    (1/C) = 1.0    → moderate penalty → balanced → often a good starting point
```

The Effect on the Decision Boundary



```
    Fits all training patients perfectly but is unstable.
    HIGH VARIANCE. Change 10 training patients and this boundary
    looks completely different.
```

L1 vs. L2 Regularisation

```
from sklearn.linear_model import LogisticRegression

# L2 (Ridge) — default, shrinks weights toward 0 but rarely to exactly 0
model_l2 = LogisticRegression(penalty='l2', C=1.0)

# L1 (Lasso) — drives unimportant feature weights to EXACTLY 0
# Built-in feature selection
model_l1 = LogisticRegression(penalty='l1', C=1.0, solver='liblinear')
```

Penalty	Effect on weights	Feature Selection?	Use when
L2 (Ridge)	Shrinks all toward 0, none become exactly 0	No	All features are somewhat relevant
L1 (Lasso)	Drives unimportant weights to exactly 0	Yes	Many features, some truly irrelevant
ElasticNet	Mix of L1 and L2	Partial	High-dimensional sparse data

3.2 K-Nearest Neighbors — K and Distance Metric

K — The Primary Hyperparameter

```
Prediction rule:
y_hat  =  mode { y_i : x_i in N_K(x) }   ← majority vote among K neighbors
```

K controls how wide the "voting constituency" is for each prediction.

```
Patient X's neighborhood for different K:

K = 1:              K = 5:              K = 15:
┌─────┐             ┌─────────┐         ┌─────────────┐
│  X  │             │  X  D   │         │  X  D  N  D │
│     │             │  D   N  │         │  D  N  D  D  N │
└─────┘             └─────────┘         └─────────────┘
Only 1 voter.       5 voters:           15 voters:
Whoever is closest  4D + 1N → Diabetic  10D + 5N → Diabetic
```

```
determines outcome.
Very noisy.
```

The bias-variance picture for K:

```
K = 1:   The boundary wraps tightly around every training point.
         Captures all local noise. OVERFIT.

K = 15:  The boundary smooths out, ignoring local noise.
         Captures the true cluster structure. SWEET SPOT.

K = 799: Essentially everyone votes. The boundary is nearly a
         straight line at the global majority ratio. UNDERFIT.
```

Practical rules for choosing K:

- Start with `K = sqrt(n_training)` as a heuristic (sqrt(800) ≈ 28)
- Always try odd K values to avoid ties in binary classification
- Use cross-validation to find the optimal K from a range like [1, 3, 5, 7, 11, 15, 21, 31, 51]

Distance Metric — The Second Hyperparameter

The choice of distance metric is often overlooked but can significantly affect KNN performance.

```
Euclidean:   d(x,y) = sqrt( Sum(x_i - y_i)² )
Manhattan:   d(x,y) = Sum( |x_i - y_i| )
Minkowski:   d(x,y) = ( Sum(|x_i - y_i|^p) )^(1/p)
             p=1 → Manhattan,  p=2 → Euclidean
```

Metric	Behaviour	Best for
Euclidean	Sensitive to large differences in any dimension	Continuous features with similar scales
Manhattan	Less sensitive to large individual differences	When outliers are present
Cosine	Measures angle, not magnitude	Text/NLP, high-dimensional sparse data

> ⚠️ All distance metrics are meaningless without **feature scaling**. A difference of 1 in `Bedrooms` and a difference of 500 in `Area_sqft` are treated the same by Euclidean distance — Standardise all features first.

3.3 SVM — C, Gamma, and Kernel Type

SVM has the most complex hyperparameter landscape of the three. Two hyperparameters — C and γ — must be tuned **jointly** because they interact.

C — Margin Hardness (Soft Margin Penalty)

```
SVM optimisation with soft margin:

Minimise:   (1/2)||w||²  +  C * Sum(slack_i)

where slack_i = how much patient i violates the margin (0 if correctly classified)
```

```
C = 0.01 (very tolerant of violations):

  Support      ←  2/||w||  →  Support
  vectors                     vectors
  class 0   |      WIDE      | class 1
            |     MARGIN     |
            |                |
    ○ ○ ●  ──────────────────  ● ○ ○
    ○ ○ ●   (some violations  |○ ● ●
             are tolerated)   |
           boundary
    Wide margin → stable boundary → generalises well
    BUT: misclassifies some training patients (the violations)


C = 10000 (very intolerant of violations):

  Support vectors
  ──────────────────────────────────
  ● ● ● ● ● ● ● ← narrow margin → ● ● ● ● ●
  ──────────────────────────────────

  Narrow margin → unstable boundary → overfits training data
  Almost zero violations on training data → HIGH VARIANCE
```

Gamma — Influence Radius (RBF Kernel)

The RBF (Radial Basis Function) kernel is the most widely used:

```
K(x_i, x_j)  =  exp(  -gamma * ||x_i - x_j||²  )
```

Gamma controls how far the "influence" of a single training patient reaches:

```
High gamma (e.g., γ = 100):
    K(x_i, x_j) decays very rapidly with distance.
    Each patient only influences predictions for points
    extremely close to it.
    Effect: Very localised decision boundary.

    Patient space:
```

```
    ○ ○ ○╭●╮○ ○      ●╭○●╭○●
    ○ ○ ○╰─╯○ ○      ●╰─●╰─●
    The boundary forms tight bubbles around each training point.
    → OVERFIT: memorises training data


Low gamma (e.g., γ = 0.001):
    K(x_i, x_j) decays very slowly with distance.
    Each patient influences predictions over a very wide area.
    Effect: Smooth, sweeping boundary across the whole space.

    Patient space:
    ○ ○ ○ ○ ○ │ ● ● ● ● ●
    ○ ○ ○ ○   │   ● ● ● ● ●
    ──────────┼──────────────
              │
            boundary
    → UNDERFIT: overly simplistic, misses complex patterns
```

The C and Gamma Interaction

This is the critical insight about SVM tuning: **C and γ cannot be tuned independently**.

	gamma LOW (broad influence)	gamma MID	gamma HIGH (narrow influence)
C HIGH (tight margin)	OVERFIT because tight margin AND broad influence partially cancel	Moderate overfitting	SEVERE OVERFIT tight margin AND narrow bubbles = memorises every point
C LOW (loose margin)	UNDERFIT because loose margin AND broad influence both push toward simplicity	Moderate underfitting	Moderate — depends on data
C MID	Moderate ↑	SWEET SPOT ✅ if gamma also well-tuned	Moderate

This is why you use Grid Search — try all (C, γ) combinations simultaneously.

3.4 Other Algorithms and Their Key Hyperparameters

Decision Tree

```
from sklearn.tree import DecisionTreeClassifier
```

```
# Key hyperparameters:
model = DecisionTreeClassifier(
    max_depth=5,              # Maximum depth of the tree
                              # Higher → more complex → overfitting
    min_samples_split=20,     # Min samples required to split a node
                              # Higher → simpler tree → underfitting
    min_samples_leaf=10,      # Min samples required at a leaf node
                              # Higher → simpler tree → underfitting
    max_features='sqrt'       # Number of features considered at each split
)
```

Hyperparameter	Low value →	High value →
max_depth	Underfit (shallow tree)	Overfit (deep, complex tree)
min_samples_split	Overfit (splits on small groups)	Underfit (few splits)
min_samples_leaf	Overfit (tiny leaf groups)	Underfit (very large leaves)

Random Forest

```
from sklearn.ensemble import RandomForestClassifier

model = RandomForestClassifier(
    n_estimators=100,         # Number of trees — more is usually better, up to a
point
    max_depth=10,             # Depth of each tree
    max_features='sqrt',      # Features considered at each split (key for
diversity)
    min_samples_leaf=5
)
```

Neural Network (sklearn MLPClassifier)

```
from sklearn.neural_network import MLPClassifier

model = MLPClassifier(
    hidden_layer_sizes=(128, 64, 32),  # Architecture: 3 layers
    learning_rate_init=0.001,          # Step size for gradient descent
    alpha=0.0001,                      # L2 regularisation strength
    dropout=0.3,                       # Fraction of neurons dropped each step
    batch_size=32,                     # Samples per gradient update
    max_iter=200                       # Number of training epochs
)
```

Learning rate deserves special attention:

```
Learning rate too high (e.g., 0.1):
    Cost oscillates — overshoots the minimum
    ╲  ╱╲  ╱╲  ╱  Loss curve goes up and down
     ╲╱  ╲╱  ╲╱

Learning rate too low (e.g., 0.000001):
    Training takes thousands of epochs to converge
    ──────────────────────────────╲──────────  (very slow descent)

Learning rate just right (e.g., 0.001):
    Smooth, steady decrease in loss
    ─────╲──────────────────────────────────
          ╲──────────────────── (clean convergence)
```

PART 4 — Cross-Validation: The Honest Measuring System

4.1 The Problem That Cross-Validation Solves

Before cross-validation, there is a fundamental dilemma:

The dilemma:

- To choose the best hyperparameter, you need to evaluate the model's performance on data it hasn't been trained on
- But the test set must stay locked until the very end — you cannot use it for tuning
- A single static validation split (e.g., use 20% of training data for validation) gives unreliable estimates

Why a single validation split is unreliable:

```
Split attempt 1 (random seed=42):
  Training: patients 1-640, Validation: patients 641-800
  Validation accuracy with C=1.0: 87%

Split attempt 2 (random seed=7):
  Training: patients 1-640 (different shuffle), Validation: patients 641-800
  Validation accuracy with C=1.0: 79%

Same hyperparameter, same algorithm, different random split → 8% different result!
```

The validation performance depends heavily on **which patients happened to end up in the validation set** — a matter of chance. If the 160 validation patients are unusually easy (or unusually hard) to classify, your estimate is biased.

4.2 K-Fold Cross-Validation — The Mechanism

K-Fold CV solves the instability problem by using **every patient for validation exactly once**.

Algorithm:

1. Divide the 800 training patients into K equal folds (K=5 → each fold has 160 patients)
2. For a given hyperparameter setting:
    - Train on folds 2,3,4,5 → validate on fold 1 → record score
    - Train on folds 1,3,4,5 → validate on fold 2 → record score
    - Train on folds 1,2,4,5 → validate on fold 3 → record score
    - Train on folds 1,2,3,5 → validate on fold 4 → record score
    - Train on folds 1,2,3,4 → validate on fold 5 → record score
3. Average the 5 scores → this is the CV score for this hyperparameter setting
4. Repeat steps 2-3 for every hyperparameter value
5. Select the hyperparameter with the highest average CV score

```
VISUALISING 5-FOLD CV ON 800 PATIENTS:

Patient blocks (each block = 160 patients):
  [  Block 1  |  Block 2  |  Block 3  |  Block 4  |  Block 5  ]

Round 1:  [ VAL ████ |  TRAIN      |  TRAIN      |  TRAIN      |  TRAIN      ]
           ↳ train on 640, validate on 160 → score = 84.2%

Round 2:  [ TRAIN    |  VAL ████   |  TRAIN      |  TRAIN      |  TRAIN      ]
           ↳ train on 640, validate on 160 → score = 86.1%

Round 3:  [ TRAIN    |  TRAIN      |  VAL ████   |  TRAIN      |  TRAIN      ]
           ↳ train on 640, validate on 160 → score = 83.7%

Round 4:  [ TRAIN    |  TRAIN      |  TRAIN      |  VAL ████   |  TRAIN      ]
           ↳ train on 640, validate on 160 → score = 85.5%

Round 5:  [ TRAIN    |  TRAIN      |  TRAIN      |  TRAIN      |  VAL ████   ]
           ↳ train on 640, validate on 160 → score = 84.9%

                              Average CV score = 84.88%
                              Std deviation    = 0.88%  ← stability measure
```

What the standard deviation tells you:

- Std = 0.88% → results are stable across folds → reliable estimate
- Std = 8% → results vary wildly → your dataset may have structural issues, or you need more data

---

4.3 Why K-Fold Is Better Than a Single Split — Statistically

```
SINGLE SPLIT (one random 20% validation set):
  Estimate is based on ONE sample of 160 patients.
  The estimate has HIGH VARIANCE — it can be 79% or 87% by chance.

5-FOLD CV:
  Estimate is the average of FIVE independent evaluations.
  By the Law of Large Numbers, the average converges to the true performance.
  The variance of the estimate is reduced by a factor of √5 ≈ 2.24.

  std_error_of_mean  =  std_of_individual_folds / sqrt(K)
                     =  0.88% / sqrt(5)
                     =  0.39%   ← much tighter estimate
```

4.4 Stratified K-Fold — Handling Class Imbalance

Regular K-Fold splits patients randomly. In our dataset, 70% are Non-Diabetic and 30% are Diabetic. With pure random splitting, one fold might accidentally contain 50% Diabetic patients and another might contain only 15%.

Stratified K-Fold ensures that each fold maintains the **same class proportion as the original dataset**:

```
Original distribution: 70% Non-Diabetic, 30% Diabetic

Regular K-Fold (can happen by chance):
  Fold 1: 55% ND, 45% D   ← unrepresentative
  Fold 2: 78% ND, 22% D
  Fold 3: 72% ND, 28% D
  Fold 4: 65% ND, 35% D
  Fold 5: 80% ND, 20% D

Stratified K-Fold (guaranteed):
  Fold 1: 70% ND, 30% D   ✅
  Fold 2: 70% ND, 30% D   ✅
  Fold 3: 70% ND, 30% D   ✅
  Fold 4: 70% ND, 30% D   ✅
  Fold 5: 70% ND, 30% D   ✅
```

```python
from sklearn.model_selection import StratifiedKFold, cross_val_score
from sklearn.linear_model import LogisticRegression

model = LogisticRegression(C=1.0)
skf = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)

scores = cross_val_score(model, X_train, y_train,
                         cv=skf, scoring='f1')

print(f"F1 per fold: {scores.round(3)}")
```

```
print(f"Mean F1:     {scores.mean():.3f}")
print(f"Std F1:      {scores.std():.3f}")
```

Always use StratifiedKFold for classification problems. For regression, use regular KFold.

4.5 5-Fold vs. 10-Fold vs. Leave-One-Out CV

The choice of K in K-Fold involves a tradeoff between:

- **Training set size per fold** (higher K → more training data → lower bias in each fold's estimate)
- **Stability of the estimate** (higher K → more folds averaged → lower variance)
- **Computational cost** (higher K → more training runs)

```
K = 5 (Five-Fold):
  Each fold trains on 80% of the training data (640/800 patients)
  Each fold validates on 20% (160 patients)
  5 training runs total
  Fast. Slightly less stable estimate.
  ✅ Standard choice for most datasets

K = 10 (Ten-Fold):
  Each fold trains on 90% of the training data (720/800 patients)
  Each fold validates on 10% (80 patients)
  10 training runs total
  Slower. More stable estimate. Less bias.
  ✅ Preferred for larger datasets or when stability matters most

K = n (Leave-One-Out CV / LOOCV):
  Each fold trains on 799 patients, validates on 1 patient
  800 training runs total
  Extremely slow. Minimum bias (almost all data used for training each time).
  Near-zero variance in the estimate.
  ✅ Only for tiny datasets (<100 samples) where you cannot afford to lose any
training data

Stratified / Repeated K-Fold:
  Run K-Fold multiple times with different random splits, average all runs.
  e.g., 5×2 CV: 2 folds, 5 different random shuffles = 10 training runs
  ✅ When you need maximum reliability and have moderate compute budget
```

Method	Training % per fold	Compute cost	Bias	Variance of estimate
5-Fold	80%	Low	Moderate	Moderate
10-Fold	90%	Moderate	Low	Low
LOOCV	~100%	Very High	Very Low	Very Low
Repeated 5-Fold	80%	High	Moderate	Very Low

4.6 Cross-Validation in Code

```
from sklearn.model_selection import (cross_val_score, StratifiedKFold,
                                     cross_validate)
from sklearn.linear_model import LogisticRegression

model = LogisticRegression(C=1.0)
cv    = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)

# Single metric
scores = cross_val_score(model, X_train, y_train, cv=cv, scoring='accuracy')
print(f"Accuracy: {scores.mean():.3f} ± {scores.std():.3f}")

# Multiple metrics at once
results = cross_validate(model, X_train, y_train, cv=cv,
                         scoring=['accuracy', 'f1', 'roc_auc'])
print(f"Accuracy: {results['test_accuracy'].mean():.3f}")
print(f"F1:       {results['test_f1'].mean():.3f}")
print(f"AUC:      {results['test_roc_auc'].mean():.3f}")
```

PART 5 — Hyperparameter Search Strategies

5.1 Manual Search — Knowing the Landscape

Before automating search, understand the rough effect of each hyperparameter so you define a sensible search range. Searching `C` from 0.00001 to 100,000 wastes compute; searching `C` from 0.001 to 100 covers the practically meaningful range.

```
Logistic Regression C:    [0.001, 0.01, 0.1, 1, 10, 100]         (log scale)
KNN K:                    [1, 3, 5, 7, 11, 15, 21, 31, 51]        (odd integers)
SVM C:                    [0.1, 1, 10, 100]                        (log scale)
SVM gamma:                [0.001, 0.01, 0.1, 1]                   (log scale)
Decision Tree max_depth:  [3, 5, 7, 10, 15, None]                 (linear scale)
```

Always search on log scale for regularisation parameters (C, lambda, alpha). The effect of changing C from 0.1 to 1 is much larger than changing C from 100 to 101. Log scale covers the relevant range efficiently.

5.2 Grid Search Cross-Validation

Grid Search evaluates **every combination** of hyperparameter values with cross-validation.

Single Hyperparameter (Logistic Regression C):

```
        C = 0.001  C = 0.01  C = 0.1  C = 1.0  C = 10   C = 100
CV Acc:   74%        79%       83%      86% ✅     84%      81%
```

Two Hyperparameters (SVM C and gamma):

```
              gamma=0.001   gamma=0.01   gamma=0.1   gamma=1.0
C = 0.1    [    72%           78%          76%          68%   ]
C = 1.0    [    80%           87% ✅         83%          74%   ]
C = 10     [    81%           85%          79%          70%   ]
C = 100    [    82%           83%          71%          62%   ]

Best combination: C=1.0, gamma=0.01 with CV accuracy = 87%
```

```python
from sklearn.model_selection import GridSearchCV
from sklearn.svm import SVC

param_grid = {
    'C':     [0.1, 1.0, 10, 100],
    'gamma': [0.001, 0.01, 0.1, 1.0],
    'kernel': ['rbf']
}

grid_search = GridSearchCV(
    estimator  = SVC(),
    param_grid = param_grid,
    cv         = StratifiedKFold(n_splits=5, shuffle=True, random_state=42),
    scoring    = 'f1',                    # optimise for F1 (medical context)
    n_jobs     = -1,                      # use all CPU cores
    verbose    = 2,                       # print progress
    refit      = True                     # refit on full training data with best
params
)

grid_search.fit(X_train, y_train)

print(f"Best parameters: {grid_search.best_params_}")
print(f"Best CV F1:      {grid_search.best_score_:.3f}")

# Evaluate on locked test set
y_pred = grid_search.predict(X_test)    # uses the refitted best model
automatically
```

Total training runs: 4 (C values) × 4 (gamma values) × 5 (folds) = **80 training runs**

Cost: For large models or datasets, Grid Search can be prohibitively expensive. This is where Random Search and Bayesian Search come in.

5.3 Random Search Cross-Validation

Instead of trying every combination, sample `n_iter` random combinations from the hyperparameter space.

```
from sklearn.model_selection import RandomizedSearchCV
from scipy.stats import loguniform, randint

param_distributions = {
    'C':     loguniform(0.001, 1000),    # sample C from log-uniform distribution
    'gamma': loguniform(0.0001, 10),     # sample gamma from log-uniform
distribution
    'kernel': ['rbf', 'linear']
}

random_search = RandomizedSearchCV(
    estimator          = SVC(),
    param_distributions= param_distributions,
    n_iter             = 50,              # try 50 random combinations
    cv                 = StratifiedKFold(n_splits=5),
    scoring            = 'f1',
    n_jobs             = -1,
    random_state       = 42,
    refit              = True
)

random_search.fit(X_train, y_train)
print(f"Best parameters: {random_search.best_params_}")
```

Why Random Search can outperform Grid Search:

```
Grid Search on C and gamma (4×4 = 16 combinations):

    gamma:  0.001  0.01   0.1    1.0
C:
0.1         ×      ×      ×      ×
1.0         ×      ×      ×      ×     Each × is evaluated
10          ×      ×      ×      ×
100         ×      ×      ×      ×

If gamma matters much more than C, you've wasted evaluations varying C
across the same 4 gamma values. You only explored 4 gamma values.

Random Search with 16 evaluations:

    gamma can take ANY value between 0.0001 and 10 (continuous)
    C can take ANY value between 0.001 and 1000 (continuous)

    16 evaluations explore 16 different gamma values
    → Much better coverage of the gamma dimension
```

```
→ For high-dimensional hyperparameter spaces, Random Search
  explores the space more efficiently than Grid Search.
```

Rule of thumb: Grid Search for ≤3 hyperparameters with small grids. Random Search for ≥3 hyperparameters or continuous ranges.

5.4 Bayesian Optimisation — The Smart Search

Grid and Random Search treat each trial independently. Bayesian Optimisation **learns from previous trials** to intelligently decide where to search next.

The idea:

1. Try a few random hyperparameter combinations (exploration)
2. Build a probabilistic model (surrogate model) of `performance = f(hyperparameters)`
3. Use the surrogate to predict which untried combination is most likely to be better
4. Try that combination → update the surrogate model → repeat

```
Trial 1:  C=1.0, γ=0.01   → F1 = 0.87   ← good
Trial 2:  C=10,  γ=0.1    → F1 = 0.79
Trial 3:  C=0.1, γ=0.001  → F1 = 0.72

Surrogate model predicts: the best region is around C=1–3, γ=0.005–0.02

Trial 4:  C=1.5, γ=0.015  → F1 = 0.89  ← even better!
Trial 5:  C=2.0, γ=0.01   → F1 = 0.88

Surrogate model converges to: C≈1.5, γ≈0.015 is the optimal region.
```

```python
# Using optuna — the most popular Bayesian optimisation library
import optuna
from sklearn.svm import SVC
from sklearn.model_selection import cross_val_score, StratifiedKFold

def objective(trial):
    C     = trial.suggest_float('C',     0.001, 1000, log=True)
    gamma = trial.suggest_float('gamma', 0.0001, 10,  log=True)

    model = SVC(C=C, gamma=gamma, kernel='rbf')
    cv    = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
    score = cross_val_score(model, X_train, y_train, cv=cv, scoring='f1').mean()
    return score

study = optuna.create_study(direction='maximize')
study.optimize(objective, n_trials=50, show_progress_bar=True)
```

```
print(f"Best params: {study.best_params}")
print(f"Best F1:     {study.best_value:.3f}")
```

When to use Bayesian Optimisation: When each training run is expensive (deep learning, large datasets) and you want to find good hyperparameters with fewer trials than Grid or Random Search.

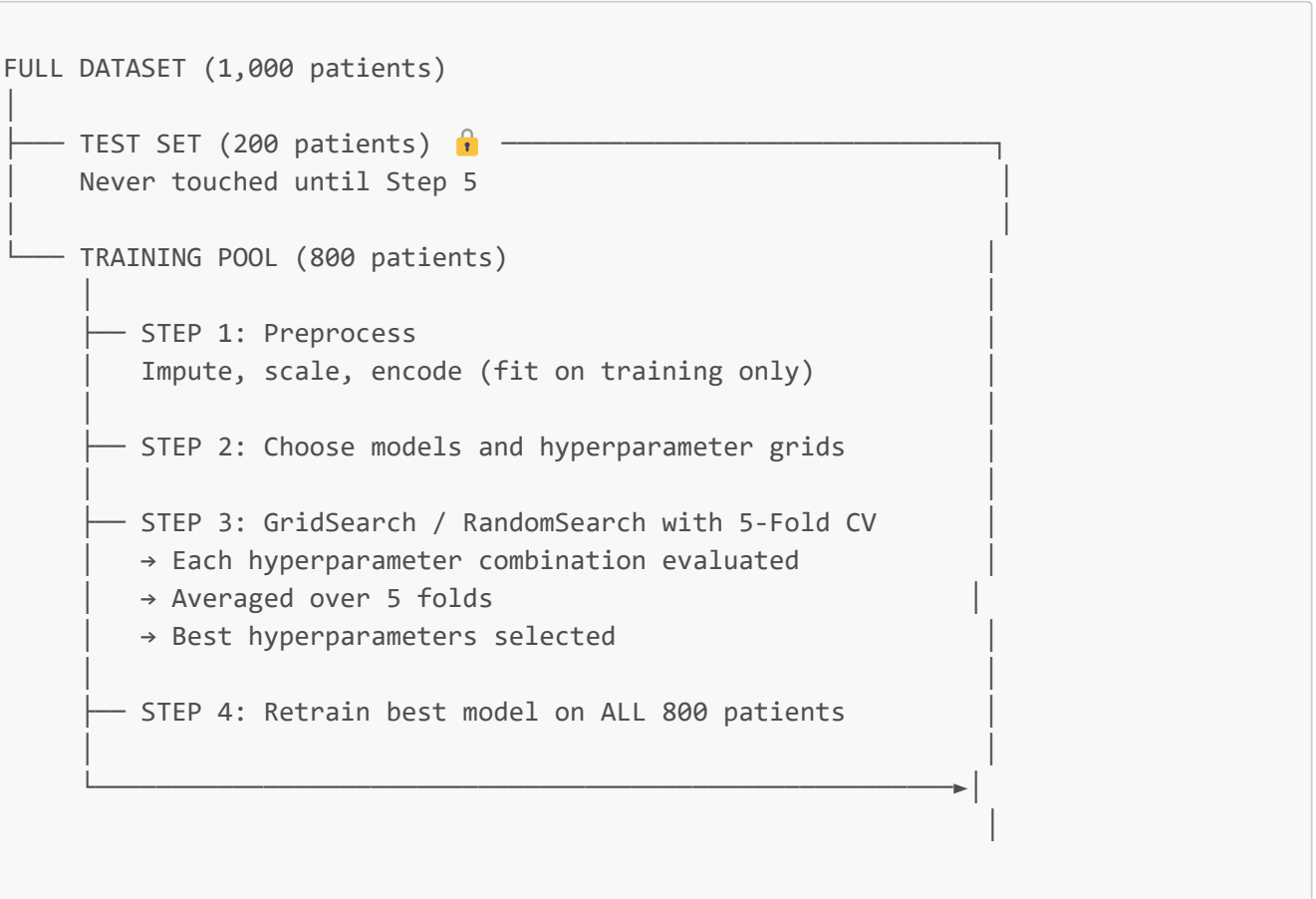
5.5 Search Strategy Comparison

Strategy	Explores	Efficiency	Best for
Grid Search	All combinations in grid	Low for high dimensions	≤3 hyperparameters, small grid
Random Search	Random samples	Better for high dimensions	≥3 hyperparameters, continuous ranges
Bayesian Optimisation	Guided by prior results	High	Expensive models (deep learning), many hyperparameters
Manual	Expert-guided specific values	Depends on expertise	Quick experiments, domain knowledge available

---

PART 6 — The Complete Tuning Pipeline End-to-End

6.1 The Right Structure — Why Order Matters



```
STEP 5: Evaluate on locked test set                                    ↵
        ONCE. Report final honest performance.
```

6.2 Full Pipeline in Code — Logistic Regression

```python
import numpy as np
import pandas as pd
from sklearn.model_selection import (train_test_split, GridSearchCV,
                                     StratifiedKFold, cross_validate)
from sklearn.preprocessing import StandardScaler
from sklearn.impute import SimpleImputer
from sklearn.linear_model import LogisticRegression
from sklearn.pipeline import Pipeline
from sklearn.metrics import classification_report, confusion_matrix

# ── Step 1: Split data ──────────────────────────────────────────
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, stratify=y, random_state=42
)

# ── Step 2: Build pipeline (no leakage guaranteed) ──────────────
pipe = Pipeline([
    ('imputer', SimpleImputer(strategy='median')),
    ('scaler',  StandardScaler()),
    ('model',   LogisticRegression(max_iter=1000))
])

# ── Step 3: Define hyperparameter grid ──────────────────────────
param_grid = {
    'model__C':       [0.001, 0.01, 0.1, 1.0, 10, 100],
    'model__penalty': ['l2'],
    'model__solver':  ['lbfgs']
}
# Note: prefix 'model__' because the model is inside a Pipeline

# ── Step 4: Grid Search with Stratified 5-Fold CV ───────────────
cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)

grid_search = GridSearchCV(
    estimator  = pipe,
    param_grid = param_grid,
    cv         = cv,
    scoring    = 'recall',          # medical: prioritise recall
    n_jobs     = -1,
    refit      = True,              # refit with best params on all 800 patients
    verbose    = 1
)

grid_search.fit(X_train, y_train)
```

```
# ── Step 5: Inspect results ──────────────────────────────
results_df = pd.DataFrame(grid_search.cv_results_)
print(results_df[['param_model__C', 'mean_test_score', 'std_test_score']]
      .sort_values('mean_test_score', ascending=False)
      .head(10))

print(f"\nBest C:      {grid_search.best_params_['model__C']}")
print(f"Best CV Recall: {grid_search.best_score_:.3f}")

# ── Step 6: Final evaluation on locked test set ──────────
y_pred = grid_search.predict(X_test)    # uses best refitted model

print("\n" + "="*50)
print("FINAL TEST SET PERFORMANCE (honest evaluation)")
print("="*50)
print(classification_report(y_test, y_pred, target_names=['Not Diabetic',
'Diabetic']))
print("Confusion Matrix:")
print(confusion_matrix(y_test, y_pred))
```

6.3 Comparing All Three Models — Full Example

```
from sklearn.svm import SVC
from sklearn.neighbors import KNeighborsClassifier

models_and_params = {
    'Logistic Regression': {
        'model': LogisticRegression(max_iter=1000),
        'params': {'C': [0.001, 0.01, 0.1, 1, 10, 100]}
    },
    'KNN': {
        'model': KNeighborsClassifier(),
        'params': {'n_neighbors': [1, 3, 5, 7, 11, 15, 21, 31],
                   'metric': ['euclidean', 'manhattan']}
    },
    'SVM': {
        'model': SVC(kernel='rbf', probability=True),
        'params': {'C':     [0.1, 1, 10, 100],
                   'gamma': [0.001, 0.01, 0.1, 1]}
    }
}

cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
best_models = {}

for name, config in models_and_params.items():
    # Build pipeline with preprocessing
    pipe = Pipeline([
        ('imputer', SimpleImputer(strategy='median')),
        ('scaler',  StandardScaler()),
```

```
        ('model',   config['model'])
    ])

    # Prefix params with 'model__'
    prefixed_params = {f'model__{k}': v for k, v in config['params'].items()}

    gs = GridSearchCV(pipe, prefixed_params, cv=cv,
                      scoring='f1', n_jobs=-1, refit=True)
    gs.fit(X_train, y_train)
    best_models[name] = gs

    print(f"\n{name}:")
    print(f"  Best params: {gs.best_params_}")
    print(f"  Best CV F1:  {gs.best_score_:.3f}")

# ── Final comparison on locked test set ──────────────────────
print("\n" + "="*60)
print("FINAL TEST SET COMPARISON")
print("="*60)
print(f"{'Model':<25} {'Accuracy':>10} {'Precision':>10} {'Recall':>8} {'F1':>6}")
print("-"*60)

from sklearn.metrics import accuracy_score, precision_score, recall_score,
f1_score

for name, gs in best_models.items():
    y_pred = gs.predict(X_test)
    print(f"{name:<25} "
          f"{accuracy_score(y_test, y_pred):>10.3f} "
          f"{precision_score(y_test, y_pred):>10.3f} "
          f"{recall_score(y_test, y_pred):>8.3f} "
          f"{f1_score(y_test, y_pred):>6.3f}")
```

Sample output:

```
============================================================
FINAL TEST SET COMPARISON
============================================================
Model                     Accuracy  Precision   Recall     F1
------------------------------------------------------------
Logistic Regression          0.865      0.844    0.813  0.828
KNN                          0.835      0.805    0.775  0.790
SVM                          0.895      0.873    0.863  0.868 ✅
```

PART 7 — Nested Cross-Validation

7.1 The Problem With Single-Level CV for Model Comparison

When you use cross-validation to both **tune hyperparameters** and **report final performance**, you introduce a subtle optimism bias:

```
BIASED APPROACH:
  CV scores: [87%, 85%, 83%, 89%, 86%] → Average = 86%
             ↑
  These are ALL used to select the best hyperparameter.
  The "86%" estimate is optimistically biased because it
  was the best score found by searching multiple settings.

Example: If you try 20 C values, the best one will look
good partly by chance. Its reported CV score is inflated.
```

Nested Cross-Validation separates the two tasks:

- **Inner loop:** Hyperparameter tuning (finds best hyperparameters)
- **Outer loop:** Performance estimation (measures generalisation honestly)

```
NESTED CV STRUCTURE:

Outer fold 1: [Test 1 | ─── Inner CV on remaining 4 folds ───]
              Tune hyperparams on inner folds.
              Evaluate tuned model on Test 1 → score₁

Outer fold 2: [──────── | Test 2 | Inner CV on remaining folds ]
              Tune hyperparams on inner folds.
              Evaluate tuned model on Test 2 → score₂

...and so on for all outer folds.

Final performance = Average(score₁, score₂, ..., scoreK)
This estimate is unbiased.
```

```
from sklearn.model_selection import cross_val_score

# Inner CV for hyperparameter tuning
inner_cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
outer_cv  = StratifiedKFold(n_splits=5, shuffle=True, random_state=0)

pipe = Pipeline([
    ('scaler', StandardScaler()),
    ('model',  LogisticRegression(max_iter=1000))
])

gs = GridSearchCV(pipe, {'model__C': [0.01, 0.1, 1, 10, 100]},
                  cv=inner_cv, scoring='f1')

# Outer loop evaluates the entire grid-search process
```

```
nested_scores = cross_val_score(gs, X_train, y_train,
                                cv=outer_cv, scoring='f1')

print(f"Nested CV F1: {nested_scores.mean():.3f} ± {nested_scores.std():.3f}")
```

When is nested CV necessary?

- When you are **reporting model performance in a research paper** — unbiased estimate is required
- When you have **very limited data** and cannot afford a separate test set
- When comparing multiple models fairly — each model gets its own inner CV tuning

When is regular CV sufficient?

- In most production settings where you have a locked test set
- When the dataset is large enough that overfitting on the hyperparameter search is negligible

---

PART 8 — Common Mistakes and How to Avoid Them

8.1 Data Leakage in Cross-Validation

The most dangerous and common mistake. Leakage occurs when information from the validation set influences the training process.

Mistake 1: Scaling before splitting

```
# WRONG — leakage
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)          # uses ALL data including validation
X_train, X_test = train_test_split(X_scaled)

# CORRECT — use Pipeline
pipe = Pipeline([('scaler', StandardScaler()), ('model', LogisticRegression())])
pipe.fit(X_train, y_train)   # scaler fits on X_train only
pipe.predict(X_test)          # scaler transforms X_test using train statistics
```

Mistake 2: Applying SMOTE before cross-validation

```
# WRONG — creates synthetic samples from the full training set
#         then splits, so synthetic samples can appear in validation
from imblearn.over_sampling import SMOTE
X_resampled, y_resampled = SMOTE().fit_resample(X_train, y_train)
# Now running CV on X_resampled is incorrect

# CORRECT — use imblearn Pipeline that applies SMOTE inside each fold
from imblearn.pipeline import Pipeline as ImbPipeline
from imblearn.over_sampling import SMOTE
```

```
pipe = ImbPipeline([
    ('smote',  SMOTE(random_state=42)),
    ('scaler', StandardScaler()),
    ('model',  LogisticRegression())
])
scores = cross_val_score(pipe, X_train, y_train, cv=cv, scoring='f1')
```

Mistake 3: Feature selection before cross-validation

```
# WRONG — feature selection sees the validation fold's data
selector = SelectKBest(k=5)
X_selected = selector.fit_transform(X_train, y_train)  # uses all X_train
# Then running CV on X_selected is leaky

# CORRECT — feature selection inside the pipeline
from sklearn.feature_selection import SelectKBest, f_classif
pipe = Pipeline([
    ('selector', SelectKBest(f_classif, k=5)),
    ('scaler',   StandardScaler()),
    ('model',    LogisticRegression())
])
scores = cross_val_score(pipe, X_train, y_train, cv=cv, scoring='f1')
```

The rule is absolute: Any transformation that learns from data (scaling, imputing, encoding, feature selection, SMOTE) must be **fitted inside each fold's training portion only** — never on the fold as a whole. The Pipeline enforces this automatically.

8.2 Using Accuracy for Imbalanced Datasets

```
# WRONG — accuracy is misleading for imbalanced data
grid_search = GridSearchCV(..., scoring='accuracy')

# If 85% of patients are Non-Diabetic:
# A model predicting everyone as Non-Diabetic gets 85% accuracy
# but has Recall = 0% for Diabetics.

# CORRECT — use task-appropriate metrics
grid_search = GridSearchCV(..., scoring='recall')      # medical screening
grid_search = GridSearchCV(..., scoring='f1')          # balanced consideration
grid_search = GridSearchCV(..., scoring='roc_auc')     # threshold-independent
```

8.3 Not Using the Test Set Exactly Once

```
# WRONG — peeking at the test set multiple times
for C in [0.01, 0.1, 1, 10]:
    model = LogisticRegression(C=C)
    model.fit(X_train, y_train)
    score = model.score(X_test, y_test)    # ← using test set for tuning!
    print(f"C={C}: test score = {score}")
# "Best" C is now overfit to the test set.

# CORRECT — test set used once, at the very end
grid_search = GridSearchCV(model, param_grid, cv=5, scoring='f1')
grid_search.fit(X_train, y_train)          # all tuning on training data
# Now — and only now — open the vault:
final_score = grid_search.score(X_test, y_test)
print(f"Final honest test F1: {final_score:.3f}")
```

8.4 Overfitting the Hyperparameters Themselves

If you search over an enormous grid, you will eventually find a combination that looks great on cross-validation by chance — especially with a small dataset.

```
100 hyperparameter combinations × 5-fold CV = 500 evaluations.
With 500 evaluations, you will find combinations that score well
purely by statistical chance (multiple comparison problem).

Symptoms:
  CV score:   91% (tuning CV says we're great)
  Test score: 74% (honest test says we're not)
  Gap of 17% → hyperparameter overfitting

Fixes:
  1. Use nested cross-validation (outer loop gives unbiased estimate)
  2. Limit the size of your hyperparameter grid
  3. Use held-out validation set in addition to cross-validation
  4. Report confidence intervals (score ± std), not just the best score
```

PART 9 — Choosing the Right Metric for Cross-Validation

9.1 The Metric You Optimise Is What You Get

A critical and often overlooked decision: **what metric do you pass to GridSearchCV's `scoring` parameter?**

The best hyperparameters under one metric are NOT necessarily the best under another.

```
C=1.0  optimised for Accuracy:  Accuracy=89%, Recall=81%, F1=85%
C=0.1  optimised for Recall:    Accuracy=84%, Recall=91%, F1=87%

In a hospital: C=0.1 is better because it catches more real diabetics,
               even though it has lower raw accuracy.
```

9.2 Metric Selection Guide

Context	Recommended Scoring	Reason
Medical diagnosis, disease screening	`recall`	Missing a real case (FN) is most costly
Spam filter, legal decisions	`precision`	False alarms (FP) are most costly
Imbalanced data, general classification	`f1`	Balances precision and recall
Model comparison, ranking	`roc_auc`	Threshold-independent, works for any cutoff
All errors equally costly, balanced data	`accuracy`	Simple and interpretable
Probabilistic outputs needed	`neg_log_loss`	Evaluates confidence, not just label
Regression tasks	`neg_mean_absolute_error` or `neg_root_mean_squared_error`	

```python
# Available scoring strings in sklearn:
# Classification: 'accuracy', 'f1', 'precision', 'recall', 'roc_auc',
#                 'f1_macro', 'f1_weighted', 'average_precision', 'neg_log_loss'
# Regression:     'neg_mean_absolute_error', 'neg_mean_squared_error',
#                 'neg_root_mean_squared_error', 'r2'

# For multiclass F1, specify the averaging strategy:
grid_search = GridSearchCV(..., scoring='f1_weighted')  # weight by class size
grid_search = GridSearchCV(..., scoring='f1_macro')     # equal weight per class
```

PART 10 — Complete Mental Model & Summary

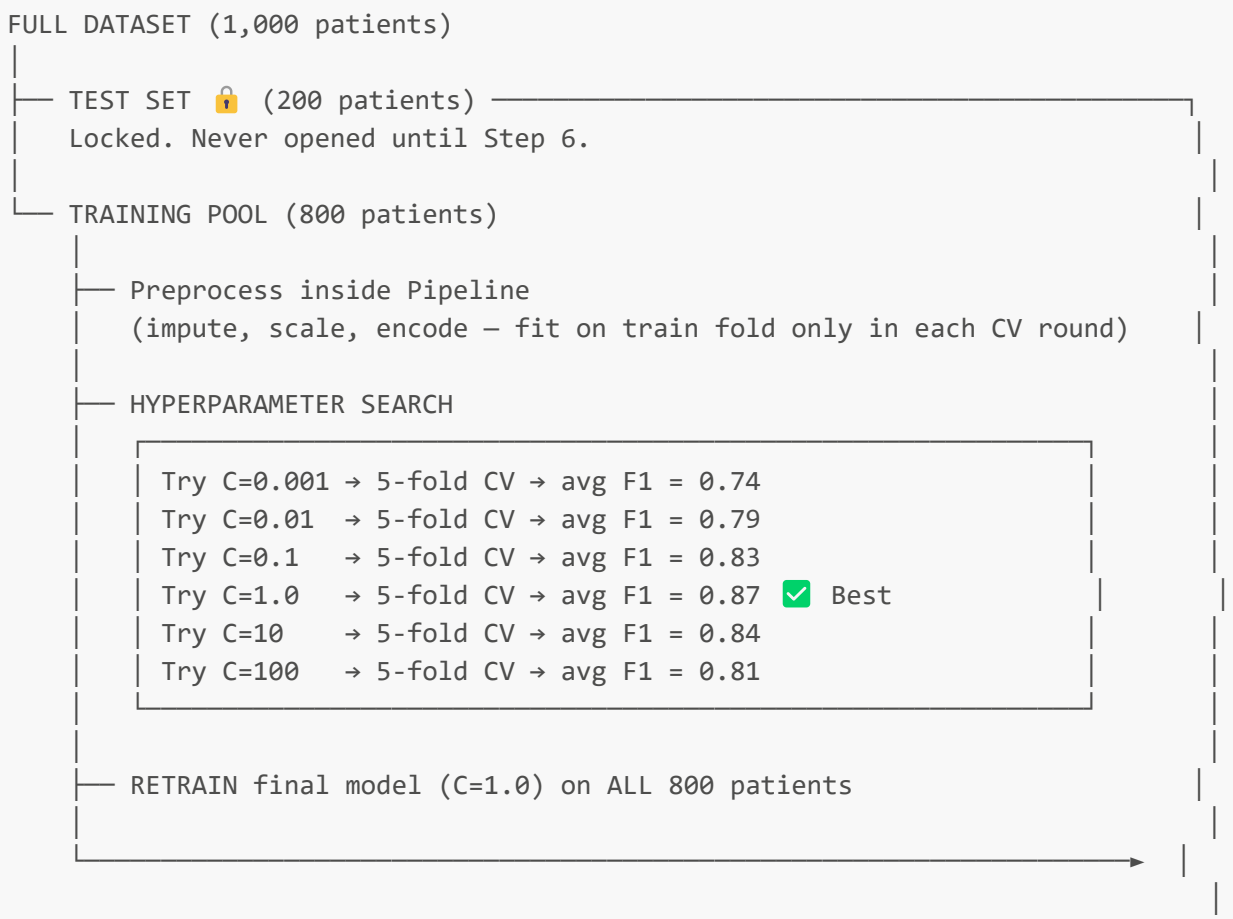
10.1 The Three Questions Tuning Answers

```
QUESTION 1: What complexity level does my specific data need?
            ↓
            Answered by: Hyperparameter grid definition

QUESTION 2: How do I measure performance honestly without using the test set?
            ↓
            Answered by: Cross-validation (the honest thermometer)

QUESTION 3: Which hyperparameter value is best according to that honest measure?
            ↓
            Answered by: Grid/Random/Bayesian Search + averaging CV scores
```

10.2 Everything Connected in One Diagram




```
WHY THE PIPELINE MATTERS:
  Without Pipeline → Scaler fits on all 800 → leakage
  With Pipeline    → Scaler fits on 640 training patients per fold → honest
```

10.3 The Mental Models — One Line Each

> **Default model** = Throwing darts blindfolded.
>
> **Hyperparameter tuning** = Someone tells you "*warmer... colder...*" as you adjust the dial.
>
> **Cross-validation** = The thermometer is calibrated and honest — not rigged.
>
> **Grid Search** = Trying every combination on the map.
>
> **Random Search** = Sampling random spots on the map — more efficient for large maps.
>
> **Bayesian Search** = Using where you've already been to intelligently decide where to go next.
>
> **Pipeline** = Your guarantee that the thermometer (cross-validation) is actually measuring what it claims to measure — not contaminated by the data it's evaluating.

10.4 Full Hyperparameter Reference Table

Algorithm	Hyperparameter	Range to Search	Scale	Effect: High →	Effect: Low →
Logistic Regression	C	[0.001 → 100]	Log	Overfit	Underfit
Logistic Regression	penalty	['l1', 'l2']	Categorical	L1: sparse weights	L2: all weights small
KNN	K (n_neighbors)	[1, 3, 5, ... 51]	Linear (odd)	Underfit	Overfit
KNN	metric	['euclidean', 'manhattan']	Categorical	—	—
SVM	C	[0.1 → 100]	Log	Overfit	Underfit
SVM	gamma (RBF)	[0.001 → 1]	Log	Overfit	Underfit
SVM	kernel	['rbf', 'linear', 'poly']	Categorical	—	—
Decision Tree	max_depth	[3, 5, 7, 10, None]	Linear	Overfit	Underfit
Decision Tree	min_samples_leaf	[1, 5, 10, 20]	Linear	Underfit	Overfit
Random Forest	n_estimators	[50, 100, 200, 500]	Linear	Better (diminishing returns)	Faster
Random Forest	max_features	['sqrt', 'log2', 0.5]	Categorical	Less diversity	More diversity
Neural Net	learning_rate	[0.0001 → 0.1]	Log	Unstable	Slow

Algorithm	Hyperparameter	Range to Search	Scale	Effect: High →	Effect: Low →
Neural Net	alpha (L2)	[0.0001 → 0.1]	Log	Underfit	Overfit
Neural Net	dropout	[0.0 → 0.5]	Linear	Underfit	Overfit

PART 11 — Interview Answer Bank

> 📖 These are the exact conceptual questions asked in interviews about hyperparameter tuning and cross-validation.

Q: What is the difference between a parameter and a hyperparameter? Give examples.

Parameters are learned automatically by the training algorithm from data — they are the values the optimiser adjusts to minimise the loss function. Examples: weights and bias in Logistic Regression, the weight vector in SVM. Hyperparameters are set before training begins and control the structure of the learning process itself — no gradient or solver can find them. Examples: C in Logistic Regression, K in KNN, max_depth in Decision Trees, learning rate in Neural Networks.

Q: Why can hyperparameters not be learned by gradient descent?

Hyperparameters operate at a meta-level — they define the rules of the learning process, not values within it. C in Logistic Regression defines the penalty term in the loss function that gradient descent minimises. You cannot include C inside the loop that C is shaping. K in KNN defines the decision mechanism entirely — there is no loss function and no gradient at all. Tree depth in Decision Trees is a stopping criterion, not a value being optimised.

Q: Why is using the test set to tune hyperparameters wrong?

The test set's purpose is to give a one-time honest estimate of how the model will perform on truly unseen data. If you use test performance to guide hyperparameter choices, you are indirectly fitting the model to the test set. The test set is no longer "unseen" — it has influenced training decisions. The reported test accuracy becomes optimistically inflated and will not represent real-world performance.

Q: Explain K-Fold Cross-Validation step by step.

Divide the training data into K equal folds. For each hyperparameter setting: train on K-1 folds, validate on the remaining fold. Rotate K times so every fold serves as validation once. Average the K validation scores — this is the CV score for this hyperparameter setting. Repeat for all hyperparameter values. Select the setting with the highest average CV score. Retrain on the full training data with that setting. Then evaluate once on the locked test set.

Q: What is the difference between 5-fold and 10-fold cross-validation? When do you use each?

5-fold trains each model on 80% of training data, 10-fold on 90%. 10-fold gives a less biased estimate (more training data per fold) and a lower variance estimate (more folds averaged), but requires twice as many training runs. Use 5-fold as the default for most problems. Use 10-fold when dataset is large and you want a more reliable estimate. Use LOOCV for very small datasets (<100 samples) where every sample matters.

Q: What is Stratified K-Fold and when must you use it?

Stratified K-Fold ensures each fold maintains the same class proportion as the full dataset. In regular K-Fold, random shuffling can create folds with very different class ratios — one fold might have 50% diabetics while another has 15%. This makes validation scores unstable and potentially misleading. Always use Stratified K-Fold for classification tasks, especially when classes are imbalanced.

Q: What is the difference between Grid Search and Random Search?

Grid Search evaluates every combination in a predefined grid. If you have 4 C values and 4 gamma values, it runs 4×4=16 combinations. Random Search samples n_iter random combinations from the defined distributions. For the same budget (16 evaluations), Grid Search covers only 4 values per dimension while Random Search can cover 16 different values of the most important dimension — making it more efficient for high-dimensional hyperparameter spaces. Grid Search is preferred for ≤3 hyperparameters with small grids; Random Search for more hyperparameters or continuous ranges.

Q: What is data leakage in cross-validation, and how does a Pipeline prevent it?

Data leakage occurs when information from the validation fold influences the training process. Common examples: fitting a StandardScaler on the full dataset before splitting (validation data's mean and std influence the scaler), applying SMOTE before splitting (synthetic samples created from the whole dataset can appear in validation). A Pipeline chains preprocessing and model together. When cross_val_score uses a Pipeline, it calls `.fit()` on each fold's training portion only and `.transform()` on the validation portion using those training-only statistics — guaranteeing no information crosses fold boundaries.

Q: What is Bayesian Optimisation in hyperparameter tuning?

Bayesian Optimisation is a sequential search strategy that builds a probabilistic surrogate model of the performance landscape from previous trials. After each evaluation, it updates the surrogate and uses it to predict which untried region is most likely to improve performance (the acquisition function). This allows it to intelligently focus evaluations in promising regions rather than searching blindly. It typically finds better hyperparameters with fewer evaluations than Grid or Random Search — making it ideal for expensive models like deep neural networks.

Q: What is nested cross-validation and when is it needed?

Nested cross-validation separates hyperparameter tuning from performance estimation using two nested loops. The inner loop runs grid search with cross-validation to find the best hyperparameters. The outer loop evaluates the full tuning process on held-out folds to estimate generalisation performance. This removes the optimism bias that comes from using the same CV to both tune and evaluate. It is needed when reporting

results in research papers, when you have very limited data and no separate test set, or when comparing multiple models fairly.

Q: You ran GridSearchCV and got a CV score of 91%, but test set score is 74%. What went wrong?

This is hyperparameter overfitting — the model has overfit to the cross-validation process itself, not just the training data. Likely causes: the hyperparameter grid was too large (100+ combinations with 5-fold CV = 500 evaluations on limited data creates the multiple comparison problem where some combination looks good by chance), the dataset is too small relative to the search space, or the CV metric was accuracy on an imbalanced dataset. Fixes: use nested cross-validation for an unbiased performance estimate, reduce the hyperparameter grid size, use a more robust metric, or gather more training data.

> **The one principle that unifies all of hyperparameter tuning and cross-validation:** "*You have two jobs: find the best model, and measure its performance honestly. These two jobs must be done with completely separate data. Cross-validation is how you do the first job without contaminating the second.*"